

Mini-Lecture: Pattern Matching

Kristopher Micinski

Syracuse University

CIS531, Fall 2025



Pattern Matching

- ◆ Hugely-useful feature, especially in functional programming w/ algebraic data
- ◆ Insight: can **always** desugar a match into a chain of ifs:
 - ◆ **If** the match pattern holds, **then** bind these variables and fire this body
 - ◆ **Else**, continue matching the rest of the patterns...
 - ◆ Default case: throw exception, no match case available...

```
(define (f x)
  (match x
    [ `(foo ,(? number? z)) (+ z 1) ]
    [ `(bar ,y ,z) (+ y z) ]))
```

```
(define (f+ x)
  (if ;; translate match pattern to big conjunction...
      (and (list? x) (equal? (length x) 2) (equal? (first x) 'foo) (number? (second x)))
      (let ([z (second x)]) (+ z 1)) ;; bind variable and fire guard
      ;; else...
      (if ;; second match pattern as big conjunction...
          (and (list? x) (equal? (length x) 3) (equal? (first x) 'bar))
          (let ([y (second x)] [z (third x)]) (+ y z))
          (error 'match-error "no possible match"))))
```

Translating (match ...)

- ◆ To translate (match e [pat0 body0] rest ...), we do...
- ◆ First, let-bind e to some value v: (let ([v e]) ...)
- ◆ Now, translate the sequence of patterns into chain of **ifs**
 - ◆ Each pattern is translated as follows:
 - ◆ Matching a literal just translates (equal? v lit)
 - ◆ `(e0 e1 ... e_{n-1}) is translated to a match of a list, of the requisite length
 - ◆ Then, the pattern for each is asserted
 - ◆ Matching a predicate (? p) simply applies the predicate p

Translating match, continued...

- ◆ Binding variables is a bit tougher... Consider [``(p (g ,x) ,y) ...`]:
 - ◆ Can't **bind** `x/y` until we know pattern matches (at least enough)
 - ◆ First test the pattern, then bind variables in pattern
- ◆ What about [``(,x ,x) ...`] — Need to ensure that all occurrences of `x` **unify**
 - ◆ Naive implementation might clobber `x`, need to recognize that `x0` is first, `x1` is second, and `x0/x1` must be equal for the pattern to fire
- ◆ Basic idea: build environment of variables, when new variable introduces that shadows old occurrence, ensure to check that binding is equal? to old value

Compiling Pattern Matching to Good Decision Trees

- ◆ Basic issue: when patterns overlap, the “serial” approach I sketched starts to become quite inefficient
- ◆ Want to leverage overlapping parts of the search space
- ◆ Need to ensure that patterns still match in first-to-last order

Compiling Pattern Matching to Good Decision Trees

Luc Maranget
INRIA — France
Luc.maranget@inria.fr

Abstract

We address the issue of compiling ML pattern matching to compact and efficient decisions trees. Traditionally, compilation to decision trees is optimized by (1) implementing decision trees as dags with maximal sharing; (2) guiding a simple compiler with heuristics. We first design new heuristics that are inspired by *necessity*, a concept from lazy pattern matching that we rephrase in terms of decision tree semantics. Thereby, we simplify previous semantic frameworks and demonstrate a straightforward connection between necessity and decision tree runtime efficiency. We complete our study by experiments, showing that optimizing compilation to decision trees is competitive with the optimizing match compiler of [Le Fessant and Maranget \(2001\)](#).

Categories and Subject Descriptors D 3. 3 [Programming Languages]: Language Constructs and Features—Patterns

General Terms Design, Performance.

Keywords Match Compilers, Decision Trees, Heuristics.

Note This version includes an appendix, which is absent from published version.

1. Introduction

Pattern matching is certainly one of the key features of functional languages. Pattern matching is a powerful high-level construct that allows programming by directly following case analysis. Cases to match are expressed as “algebraic” patterns, *i.e.* ordinary terms. Definitions by pattern matching are roughly similar to term rewriting rules: a series of rules is defined; and execution is performed on *subject* values by finding a rule whose left-hand side is matched by the value. With respect to plain term rewriting ([Terese 2003](#)), the semantics of ML simplifies two issues. First, matches are always attempted at the root of the subject value. Secondly, the matched rule is unique, thanks to the textual priority scheme.

Any ML compiler translates the high-level pattern matching definitions into low-level tests, organized in *matching automata*. Matching automata fall in two categories: *decision trees* and *backtracking automata*. Compilation to backtracking automata has been introduced by [Augustsson \(1985\)](#). The primary advantage of the technique is a linear guarantee for code size. However, backtracking automata may backtrack. Therefore, they may scan subterms

more than once. As a result, backtracking automata are potentially inefficient at runtime. The optimizing compiler of [Le Fessant and Maranget \(2001\)](#) alleviates this problem.

In this paper we study compilation to decision trees. The primary advantage of decision trees is that they never test a given subterm of the subject value more than once (and their primary drawback is potential code size explosion). We aim to refine naive compilation to decision trees, and to compare the output of the resulting optimizing compiler with optimized backtracking automata.

Compilation to decision trees is sensitive to the testing order of subject value subterms. The situation can be explained by the example of a human programmer attempting to translate a ML program into a lower-level language without pattern matching. Let f be the following function¹ that takes three boolean arguments:

```
let f x y z = match x,y,z with
| _,F,T -> 1
| F,T,_ -> 2
| _-,F -> 3
| _-,T -> 4
```

Where T and F stand for true and false, respectively.

Apart from preserving ML semantics (*e.g.* $f F T F$ should evaluate to 2), the game has one rule: never test x , y or z more than once. A natural idea is to test x first, *i.e.* to write:

```
let f x y z = if x then fT__ y z else fF__ y z
```

Where functions $fT_$ and $fF_$ are defined by pattern matching:

```
let fT__ y z = match y,z with
| F,T -> 1
| _,F -> 3
| _,T -> 4
let fF__ y z = match y,z with
| F,T -> 1
| T,_ -> 2
| _,F -> 3
| _,T -> 4
```

The matchings above are built from the initial matching, by selecting the rows that still can be matched once the value of x is known.

Compilation goes on by considering y and z , resulting in the following low-level $f1$:

```
let f1 x y z =
if x then
  if y then
    if z then 4 else 3
  else
    if z then 1 else 3
else
  if y then 2
  else
    if z then 1 else 3
```

We can do a little better, by introducing a local function definition to share the common subexpression $\text{if } z \text{ then } 1 \text{ else } 3$.

¹ We use OCaml syntax.

